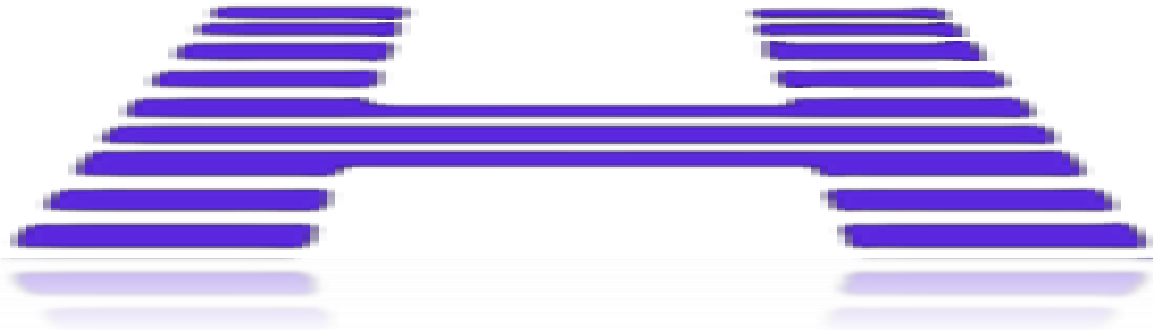




Mô hình truyền thông điệp

Hà nội, 1/2014

Đại học Bách khoa Hà Nội

Nguyễn Hữu Đức

Viện CNTT & TT

{ducnh@soict.hust.edu.vn}

Nội dung bài học

Tổng quan về mô hình truyền thông điệp

Định danh tiến trình

Phương thức truyền thông

Các giao thức sử dụng mô hình truyền thông
điệp

Đặc tả giải thuật truyền thông điệp

Giới thiệu MPI

Mô hình truyền thông điệp (Message Passing Model)



Mô hình truyền thông điệp: Phương thức giao tiếp giữa các tiến trình thông qua việc gửi thông điệp

- Hai vấn đề quan trọng trong kỹ thuật truyền thông điệp
 - Định danh các tiến trình
 - Tường minh
 - Suy diễn từ hệ thống
 - Phương thức truyền thông điệp
 - Có chờ hay không chờ khi gửi?
 - Thứ tự thông điệp có bảo toàn không?
 - Xử lý các tình huống ngoại lệ thế nào?

- Định danh tường minh

- Tiến trình gửi và tiến trình nhận chỉ định tiến trình đối tác trong câu lệnh gửi/nhận

```
send (<destination_process>, <message_length>, <message_address>);  
receive (<source_process>, <message_area>);
```

- Suy diễn định danh

- Tiến trình gửi và tiến trình nhận không chỉ định đối tác trong ngữ cảnh các đối tác này có thể xác định được

- Mailbox: hộp thư cho việc nhận thông điệp từ các tiến trình
 - Có một định danh duy nhất
 - Thường được tạo ra bởi một tiến trình chủ
 - Chỉ có tiến trình chủ mới được phép nhận thông điệp trong hộp thư
 - Các tiến trình khác có thể gửi thông điệp vào hộp thư nếu biết định danh của nó

- Gửi có chờ (blocking send)

- Tiến trình gửi chờ cho đến khi giao tiếp hoàn tất trước khi thực hiện các công việc khác
- Đơn giản hóa vấn đề xử lý tương tranh

- Gửi không chờ (non-blocking send)

- Tiến trình gửi ngay lập tức thực hiện các công việc tiếp theo mà không cần chờ đến khi giao tiếp được hoàn tất

- Các tiến trình nhận chủ yếu yêu cầu chờ cho tới khi thông điệp được chuyển giao
- Hệ thống thường tổ chức các bộ đệm cho các thông điệp gửi đi và nhận về

- Một số trường hợp ngoại lệ có thể phát sinh
 - Định danh tiến trình đích trong lệnh gửi không tồn tại
 - Định danh tiến trình nguồn trong lệnh nhận không tồn tại
 - Tràn bộ đệm
 - Không có thông điệp khi nhận
 - Khóa chết

- Simple mail transfer protocol (SMTP)
- Remote Procedure Call (RPC)
- Parallel Virtual Machine (PVM)
- Message Passing Interface (MPI)

Đặc tả giải thuật truyền thông điệp

- Các lệnh:

send $\langle M \rangle$ **to** p
receive $\langle M \rangle$ **from** p
terminate

```

init_ring_broadcast() {
    send token to successor
    receive token from predecessor
    terminate
}

```

```

ring_broadcast() {
    receive token from predecessor
    send token to successor
    terminate
}

```

- Độ phức tạp truyền thông** = số lượng thông điệp gửi bởi giải thuật (O-notation)

Chuẩn MPI (Message Passing Interface)



<http://www.mpi-forum.org>

- Chuẩn MPI là kết quả sau hơn 2 năm thảo luận của MPI Forum, 1 nhóm gồm khoảng 60 người từ 40 tổ chức khác nhau đại diện cho những nhà phân phối các hệ thống song song, những phòng thí nghiệm quốc gia và những trường đại học danh tiếng
- MPI là một thư viện các hàm có thể chèn vào mã nguồn để truyền dữ liệu giữa các tiến trình

- Chuẩn MPI-1:
 - Xác định quy cách đặt tên, quá trình thực hiện và kết quả trả về của các hàm/chương trình con trong thư viện. Mọi hàm/chương trình con trong thư viện đều phải tuân theo những quy tắc trên, điều này đảm bảo khả năng khả chuyển của các chương trình viết theo chuẩn MPI khi biên dịch và chạy trên các kiến trúc máy tính, hệ điều hành khác nhau.
 - Chi tiết cài đặt của các hàm/chương trình con trong thư viện là do những nhà cung cấp quyết định (vd: trường Đại học Indiana, Hoa Kỳ phát triển LAM/MPI, MPICH, OPENMPI,...)
 - Quá trình cài đặt các hàm/chương trình con trong thư viện phải tương thích với hầu hết các kiến trúc máy tính, hệ điều hành phổ biến.
- Năm 1994: MPI-1.0; hiện nay mới nhất là MPI-2.1
 - Chuẩn MPI-2.1 định nghĩa thêm các định tuyến nâng cao: vào ra song song, truyền thông 1 phía, ...
 - Tuy nhiên chưa được hỗ trợ nhiều

Khi nào dùng và không dùng MPI

- MPI dùng khi cần viết chương trình:
 - Đoạn mã song song có tính khả chuyển thông qua các platform khác nhau
 - Cần đạt hiệu năng cao
 - Cần viết thư viện song song
- Khi nào không nên dùng MPI:
 - Có thể đạt hiệu năng bằng cách dùng các hàm song song sẵn có của thư viện Fortran hoặc của OpenMP
 - Đã có các thư viện song song viết bằng MPI, như thư viện toán học song song
 - Không cần đoạn mã nào chạy song song

• Ngôn ngữ C

Định nghĩa các
prototype:
Hàm, macro,
hằng,....



```
#include <mpi.h>
```

Khởi tạo môi
trường MPI



```
main(int argc, char* argv[])
{
    intitation of variable;
    MPI_Init(&argc, &argv);
    ....
    ...
    ...

```

- Dọn dẹp dữ
liệu MPI
- Hủy bỏ các
hàm đang
chạy



```
MPI_Finalize();
}
```

Các đặc điểm cơ bản của CT truyền thông điệp

- Bao gồm nhiều instance của 1 chương trình tuần tự.
- Các instance truyền thông với nhau qua các định tuyến trong thư viện MPI
- Các định tuyến được chia thành:
 - Khởi tạo, quản lý và ngắt quá trình truyền thông
 - Truyền thông giữa các cặp tiến trình(truyền thông điểm – điểm)
 - Truyền thông giữa các nhóm nhóm(truyền thông công cộng)
 - Tạo các loại dữ liệu có cấu trúc riêng
 - Thao tác trên communicator

Quy ước đặt tên

- Tất cả phần tử trong MPI (định tuyến, biến, hằng, ...) đều bắt đầu bằng từ MPI_
- Tên định tuyến:
 - MPI_Xxxxxx(tham số,...)
 - MPI_Init(\$argc, \$args)
- Tên hằng:
 - MPI_XXXXX
 - MPI_COMM_WORLD, MPI_REAL

Định tuyến MPI và giá trị trả về

- Các định tuyến MPI được thực thi như hàm trong C
- Giá trị trả về của định tuyến là 1 số nguyên (integer) cho biết trạng thái kết thúc của hàm (lỗi hay thành công)

```
int err;  
err = MPI_Init(&argc, &argv);  
if (err == MPI_SUCCESS)  
{  
  
}
```

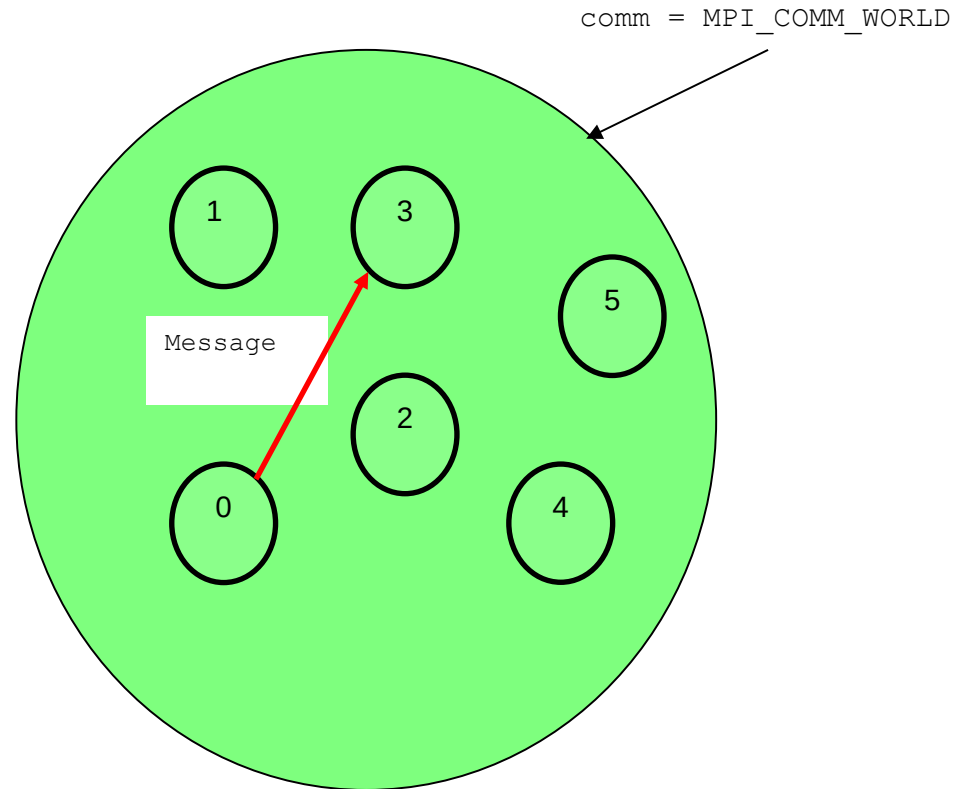
Các kiểu dữ liệu của chuẩn MPI

- Kiểu dữ liệu khai báo là `MPI_Datatype`
- Ngoài ra còn có các kiểu dữ liệu dẫn xuất từ kiểu dữ liệu chuẩn

Kiểu dữ liệu MPI	Kiểu dữ liệu tương ứng trong C
<code>MPI_CHAR</code>	<code>signed char</code>
<code>MPI_SHORT</code>	<code>signed short int</code>
<code>MPI_INT</code>	<code>signed int</code>
<code>MPI_LONG</code>	<code>signed long int</code>
<code>MPI_UNSIGNED_CHAR</code>	<code>unsigned char</code>
<code>MPI_UNSIGNED_SHORT</code>	<code>unsigned short int</code>
<code>MPI_UNSIGNED</code>	<code>unsigned int</code>
<code>MPI_UNSIGNED_LONG</code>	<code>unsigned long int</code>
<code>MPI_FLOAT</code>	<code>float</code>
<code>MPI_DOUBLE</code>	<code>double</code>
<code>MPI_LONG_DOUBLE</code>	<code>long double</code>
<code>MPI_PACKED</code>	<code>(none)</code>
<code>MPI_BYTE</code>	<code>(none)</code>

Communicator

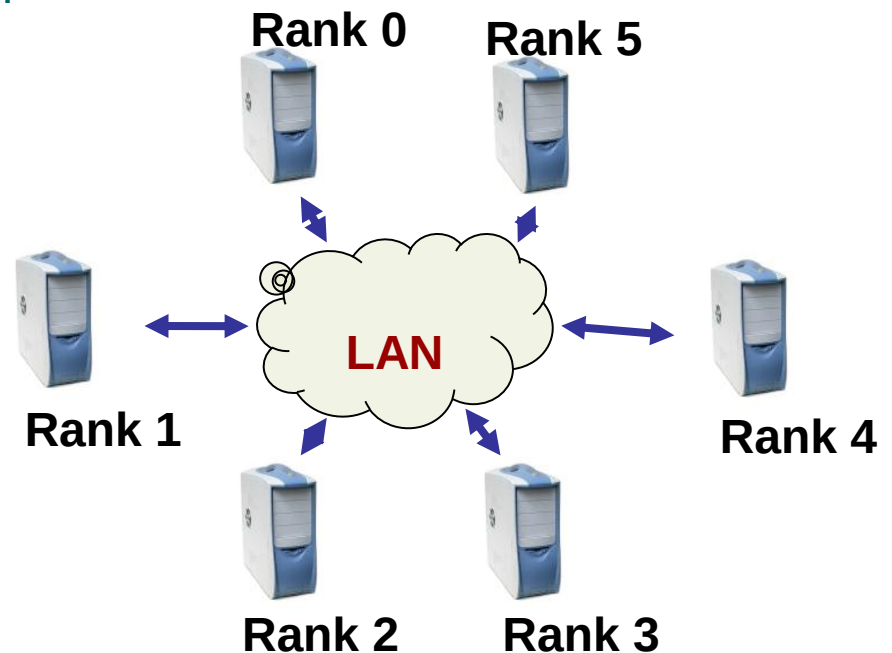
- Communicator: Một nhóm các tiến trình có thể truyền thông với nhau
- Một tiến trình có thể thuộc nhiều Communicator
- Kiểu dữ liệu biểu diễn Communicator: MPI_Comm
- MPI_Comm comm: xác định communicator của node gửi và nhận



- MPI_COMM_WORLD: Mọi tiến trình đều thuộc communicator này

Rank

- Mỗi tiến trình trong 1 communicator có 1 định danh, gọi là Rank, đánh số bắt đầu từ 0.
- Một tiến trình có thể các rank khác nhau khi thuộc về các communicator khác nhau



- `int MPI_Init()`
 - Khởi tạo tham số cho môi trường MPI
- `int MPI_Comm_rank(MPI_Comm comm, int *rank)`
 - Trả về rank của tiến trình
- `int MPI_Comm_size(MPI_Comm comm, int *size);`
 - Trả về số tiến trình trong comm
- `int MPI_Finalize()`
 - Giải phóng dữ liệu, ngắt các định tuyến MPI

Cài đặt môi trường thử nghiệm

- Sử dụng hệ thống song song sẵn có
 - Không đòi hỏi cấu hình, bảo trì, thay đổi máy tính cá nhân
 - Tập trung vào viết chương trình
 - Số lượng các nút tính toán nhiều
 - Môi trường song song thực sự
 - Hệ thống bkluster
 - Địa chỉ: bkluster.hut.edu.vn
 - Liên hệ người quản trị để lấy tài khoản đăng nhập
- Tự cài đặt môi trường song song trên máy cá nhân
 - Số lượng CPU ít
 - Đòi hỏi công sức cài đặt, bảo trì

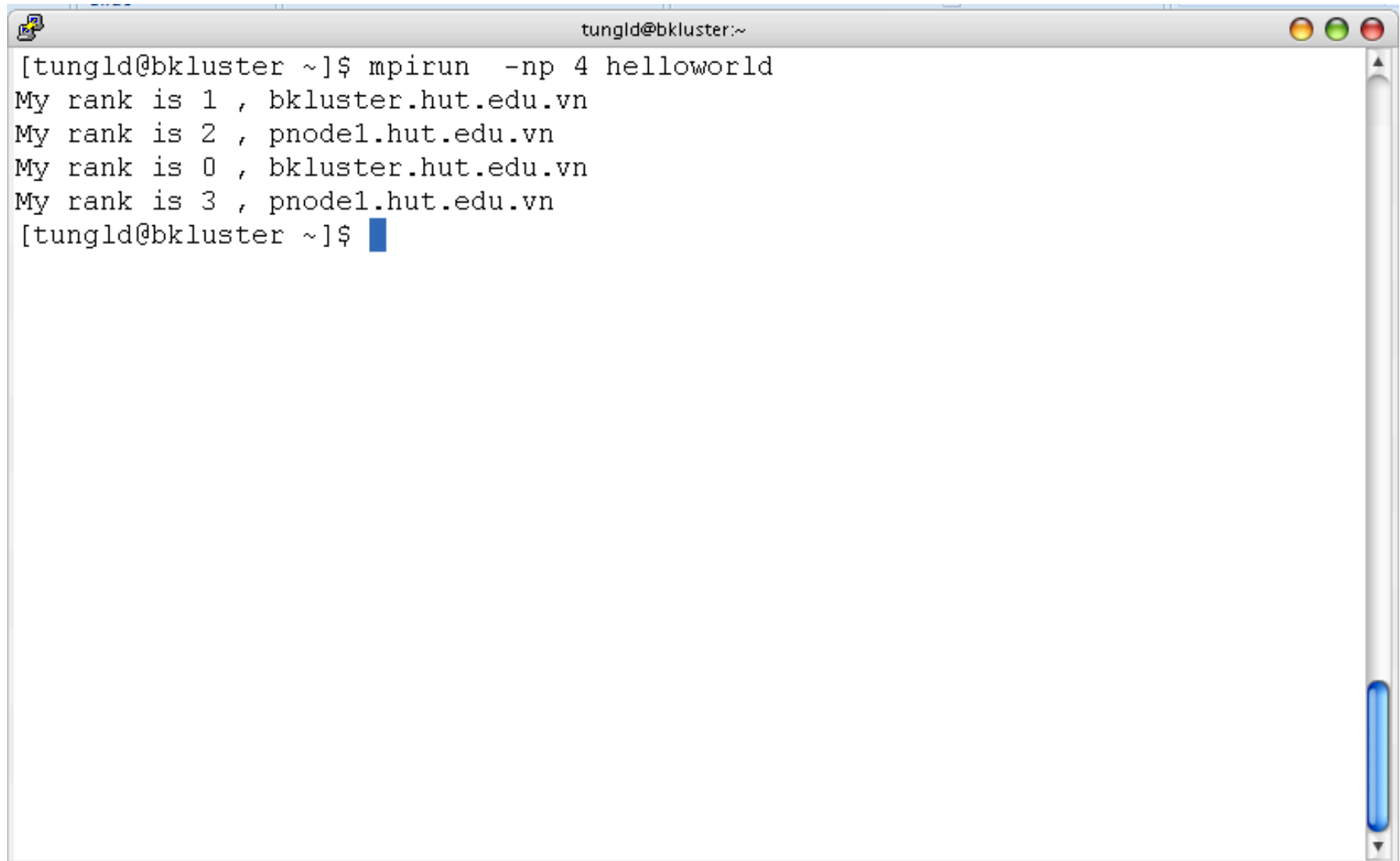
Viết chương trình helloworld.c

```
1. #include <stdio.h>
2. #include <unistd.h>
3. #include <mpi.h>
4. int main(int argc, char* argv[])
5. {
6.     int size, rank;
7.     char  hostname[50];
8.     // Khoi tao tham so cho moi truong MPI
9.     MPI_Init(&argc, &argv);
10.    // Lay ve kich thuoc pommunicator
11.    MPI_Comm_size(MPI_COMM_WORLD, &size);
12.    // Lay ve so hieu rank
13.    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
14.    // Lay ve hostname
15.    gethostname(hostname, 50);
16.    printf("My rank is %d , %s\n", rank, hostname);
17.    MPI_Finalize();
18.    return 0;
19. }
```


- Dừng lệnh
 - `mpicc helloworld.c -o helloworld`
 - Biên dịch mã nguồn ***helloworld.c*** thành chương trình chạy có tên `helloworld`

Chạy chương trình helloworld

- Chạy chương trình
 - `mpirun -np 4 helloworld`
 - Chạy chương trình hello với 4 tiến trình
- Xem chương trình có thực sự chạy không:
 - Mở 1 terminal mới
 - Gõ lệnh: `ps -aux | grep tenchuongtrinh`
 - `ps -aux | grep helloworld`
- Tắt môi trường lam/mpi
 - `lamhalt`



```

tungld@bkluster:~
[tungld@bkluster ~]$ mpirun -np 4 helloworld
My rank is 1 , bkluster.hut.edu.vn
My rank is 2 , pnode1.hut.edu.vn
My rank is 0 , bkluster.hut.edu.vn
My rank is 3 , pnode1.hut.edu.vn
[tungld@bkluster ~]$

```